

AI Recipe Recommendation Engine

```
!pip install -q pandas scikit-learn sentence-transformers gradio
```

```
import pandas as pd
import numpy as np
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import gradio as gr
```

```
recipes = pd.DataFrame({

    "Recipe": [
        "Veg Pasta",
        "Chicken Curry",
        "Paneer Salad",
        "Omelette",
        "Fruit Smoothie",
        "Veg Fried Rice",
        "Grilled Fish",
        "Tomato Soup"
    ],

    "Ingredients": [
        "pasta tomato cheese onion garlic",
        "chicken onion tomato garlic spices",
        "paneer cucumber tomato lettuce",
        "egg onion cheese pepper",
        "banana milk honey apple",
        "rice carrot beans onion soy sauce",
        "fish lemon garlic butter",
        "tomato onion garlic cream"
    ],

    "Diet": [
        "vegetarian",
        "non-vegetarian",
        "vegetarian",
        "non-vegetarian",
        "vegetarian",
        "vegetarian",
        "non-vegetarian",
        "vegetarian"
    ],

    "Skill": [
        "beginner",
        "intermediate",
        "beginner",
        "beginner",
        "beginner",
        "intermediate",
        "advanced",
        "beginner"
    ],

    "Calories": [
        450,
        650,
        300,
        250,
        180,
        400,
        500,
        220
    ],

    "CookingTime": [
        "20 mins",
        "45 mins",
        "15 mins",
        "10 mins",
        "5 mins",
        "30 mins",
        "40 mins",
        "15 mins"
    ]
})
```

})

recipes

	Recipe	Ingredients	Diet	Skill	Calories	CookingTime	
0	Veg Pasta	pasta tomato cheese onion garlic	vegetarian	beginner	450	20 mins	
1	Chicken Curry	chicken onion tomato garlic spices	non-vegetarian	intermediate	650	45 mins	
2	Paneer Salad	paneer cucumber tomato lettuce	vegetarian	beginner	300	15 mins	
3	Omelette	egg onion cheese pepper	non-vegetarian	beginner	250	10 mins	
4	Fruit Smoothie	banana milk honey apple	vegetarian	beginner	180	5 mins	
5	Veg Fried Rice	rice carrot beans onion soy sauce	vegetarian	intermediate	400	30 mins	
6	Grilled Fish	fish lemon garlic butter	non-vegetarian	advanced	500	40 mins	
7	Tomato Soup	tomato onion garlic cream	vegetarian	beginner	220	15 mins	

Next steps: [Generate code with recipes](#) [New interactive sheet](#)

```
model = SentenceTransformer('all-MiniLM-L6-v2')

recipe_embeddings = model.encode(recipes["Ingredients"].tolist())

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
modules.json: 100% 349/349 [00:00<00:00, 30.6kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 11.4kB/s]

README.md: 10.5k/? [00:00<00:00, 830kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 6.20kB/s]

config.json: 100% 612/612 [00:00<00:00, 63.9kB/s]

model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 452MB/s]

Loading weights: 100% 103/103 [00:00<00:00, 383.02it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+--
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100% 350/350 [00:00<00:00, 35.2kB/s]

vocab.txt: 232k/? [00:00<00:00, 15.4MB/s]

tokenizer.json: 466k/? [00:00<00:00, 22.9MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 11.4kB/s]

config.json: 100% 190/190 [00:00<00:00, 17.8kB/s]
```

```
def recommend_recipe(user_ingredients, diet, skill):

    # Convert user input into embedding
    user_embedding = model.encode([user_ingredients])

    # Similarity score
    similarities = cosine_similarity(user_embedding, recipe_embeddings)[0]

    recipes["Similarity"] = similarities

    # Filter by diet + skill
    filtered = recipes[
        (recipes["Diet"] == diet) &
        (recipes["Skill"] == skill)
    ]

    if filtered.empty:
        return "No matching recipes found."
```

```

# Sort by similarity
filtered = filtered.sort_values(
    by="Similarity",
    ascending=False
)

# Top 3 recipes
top_recipes = filtered.head(3)

output = ""

for _, row in top_recipes.iterrows():

    output += f"""
🍽️ Recipe: {row['Recipe']}

🥬 Ingredients:
{row['Ingredients']}

🔥 Calories:
{row['Calories']}

🕒 Cooking Time:
{row['CookingTime']}

-----
"""

return output

```

```

days = [
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday"
]

def generate_meal_plan(diet, skill):

    filtered = recipes[
        (recipes["Diet"] == diet) &
        (recipes["Skill"] == skill)
    ]

    if len(filtered) == 0:
        return "No recipes available."

    meal_plan = ""

    selected_recipes = filtered.sample(
        n=min(7, len(filtered)),
        replace=True
    )

    for day, (_, row) in zip(days, selected_recipes.iterrows()):

        meal_plan += f"""
📅 {day}

🍽️ Meal: {row['Recipe']}
🔥 Calories: {row['Calories']}
🕒 Time: {row['CookingTime']}

-----
"""

    return meal_plan

```

```

def generate_shopping_list(diet, skill):

    filtered = recipes[
        (recipes["Diet"] == diet) &
        (recipes["Skill"] == skill)
    ]

    ingredients = []

```

```

for item in filtered["Ingredients"]:

    split_items = item.split()

    ingredients.extend(split_items)

unique_ingredients = sorted(set(ingredients))

shopping_list = "\n".join(
    [f"🛒 {item}" for item in unique_ingredients]
)

return shopping_list

```

```

# Recipe Recommendation Interface
recipe_interface = gr.Interface(

    fn=recommend_recipe,

    inputs=[
        gr.Textbox(
            label="Ingredients",
            placeholder="tomato onion cheese"
        ),

        gr.Dropdown(
            ["vegetarian", "non-vegetarian"],
            label="Diet Preference"
        ),

        gr.Dropdown(
            ["beginner", "intermediate", "advanced"],
            label="Cooking Skill"
        )
    ],

    outputs="text",

    title="🍳 Recipe Recommendation Engine"
)

# Weekly Meal Planner Interface
meal_interface = gr.Interface(

    fn=generate_meal_plan,

    inputs=[
        gr.Dropdown(
            ["vegetarian", "non-vegetarian"],
            label="Diet Preference"
        ),

        gr.Dropdown(
            ["beginner", "intermediate", "advanced"],
            label="Cooking Skill"
        )
    ],

    outputs="text",

    title="🍽️ Weekly Meal Planner"
)

# Shopping List Interface
shopping_interface = gr.Interface(

    fn=generate_shopping_list,

    inputs=[
        gr.Dropdown(
            ["vegetarian", "non-vegetarian"],
            label="Diet Preference"
        ),

        gr.Dropdown(
            ["beginner", "intermediate", "advanced"],
            label="Cooking Skill"
        )
    ],

```

```
outputs="text",

title="🛒 Shopping List Generator"
)

# Combine all tabs
app = gr.TabbedInterface(

    [
        recipe_interface,
        meal_interface,
        shopping_interface
    ],

    [
        "Recipe Recommender",
        "Meal Planner",
        "Shopping List"
    ]
)

app.launch()
```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share`

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()

* Running on public URL: <https://6f16fb6bc005e3c8cd.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the

[Recipe Recommender](#)

[Meal Planner](#)

[Shopping List](#)



Recipe Recommendation Engine

Ingredients

tomato onion cheese

Diet Preference

vegetarian

Cooking Skill

beginner

output

Flag

Clear

Submit

Use via API · Built with Gradio · Settings

Next steps: [🚀 Deploy to Cloud Run](#)